

QA Engineer — Take-Home Assignment

Test Design & Automation | Finance Application

Submission: Share a GitHub repository link via email

Context

This is a short, practical assignment to understand how you approach testing in a finance domain. We are not looking for a large or perfect submission — we want to see how you think, structure your work, and communicate your decisions.

Application to Test

Choose any finance-related web application. Here are some public options you can use — pick one of these or any other finance app you prefer:

- **OrangeHRM** – demo.orangehrm.com (HR + payroll workflows)
- **Firefly III** – github.com/firefly-iii/firefly-iii (personal finance manager, can run locally via Docker)
- **Actual Budget** – github.com/actualbudget/actual (open-source budgeting app)
- **Splitwise Demo / Money Manager apps** – any publicly accessible expense tracker
- **Reqres.in or JSONPlaceholder** – if you want a clean REST API to test against for the API section

Note: The app choice is entirely yours. Mention the app name and URL in your README.

Task 1 — Manual Test Cases

Pick one core module of the finance app (e.g. Login, Add Transaction, Budget Summary, Transfer Funds) and write **6–8 test cases** for it.

Your test cases should cover:

- At least 2 positive scenarios (valid inputs, happy path)
- At least 3 negative scenarios (invalid amount, missing fields, unauthorized access)
- At least 1 boundary value (e.g. transaction amount = 0, negative value, max limit)

Format — use a simple table or bullet list in your markdown file:

- TC ID
- Test description
- Steps
- Expected result
- Priority (High / Medium / Low)

Save this as `test-cases/manual-test-cases.md` in your repo.

Task 2 — UI Automation

Automate **2–3 of your test cases** from Task 1 using any framework you are comfortable with (Selenium, Playwright, Cypress, etc.).

Keep it simple — we are looking for:

- Page Object Model (POM) structure — page classes separate from test files

- At least one positive and one negative test automated
- Meaningful assertions — verify actual text, element state, or URL, not just that no error occurred
- No `Thread.sleep()` or hard-coded waits — use explicit waits

Place code in `ui-tests/` folder. State your framework and language in the README.

Task 3 — API Automation

Automate **2–3 API test scenarios** for any REST endpoint in the app (or use a public finance-adjacent API like Reqres.in or JSONPlaceholder if the app doesn't expose one). Use any framework you are comfortable with — RestAssured, Playwright API testing, Axios, Supertest, requests (Python), etc.

Cover at least:

- One successful request — validate status code and key response fields
- One failure case — missing field, bad input, or unauthorized request

Good to have (not mandatory):

- Base URL in a config file, not hardcoded in test
- Response schema or field-type validation

Place code in `api-tests/` folder.

README — Required

Your README.md should briefly cover:

- Which app you chose and why
- Framework and language used
- How to run the tests (setup steps)
- Any assumptions or limitations

A few clear sentences per point is enough — no need for a long write-up.

What We Look For

- **Test thinking:** Do your manual test cases cover realistic finance scenarios?
 - **Code structure:** Is the POM followed? Is the code readable without explanation?
 - **Assertions:** Are you verifying behaviour, not just execution?
 - **Commit history:** A few commits as you work — not one giant commit at the end
 - **Honesty:** It is fine to note what you skipped and why
-

Submission

- Make your GitHub repository **public** before sharing
- Reply to the assignment email with your repo link
- If you have any questions before starting, feel free to reach out